ELEG 55903: Programming for Power Electronics

Homework #7

Tyler Cash

ELEG 4593

HW #7

13 March 2026

Abstract:

In this homework assignment we will create a Serial Echo Program implementation in VHDL. The serial echo program operates by repeating the character input to the character output.

Table of Contents:

List of Figures:

Overview:

This project involves implementing a Serial Echo Program in VHDL with the following inputs and outputs:

Input: RX

Output: TX

To implement this and synchronize the input/output, we make use of the internal oscillator and PLL to generate the clock and the reset.

Modules:

For this homework assignment, we did not require the use of many modules, only two in fact. We used a standard counter and a standard FIFO, to process each character in and out, echoing the input back to the output.

States:

To implement a Serial Echo Program, the key is in the states. There are four states for the read and write process. For the Read side, the first state, S0, is when it is just waiting for data to be transmitted via the start bit, then S1 is when we enable the receiver to start reading data. In S2, we are waiting for the read process to finish, and then finally in S3 we push the byte of data into the FIFO. The Write side is very similar, in S0, it is checking if there is data in the FIFO, then in S1 it loads the transmit byte. In S2 it starts transmitting and shifting bits, and finally in S3 it waits for the transmission to finish. We then use a sync state function to synchronize the sending and receiving of data

Conclusion:

In conclusion, we have properly implemented the Serial Echo Program in VHDL, as well as an appropriate test bench, demonstrating the successful implementation. The deliverables are in Figure 1 and Figure 2, the waveform and the serial interface, as well as the appendix containing the code and the test bench code. The waveform in Figure 1 is considerably larger than the provided one for comparison, I had to do some debugging and wanted to look at all of the signals.

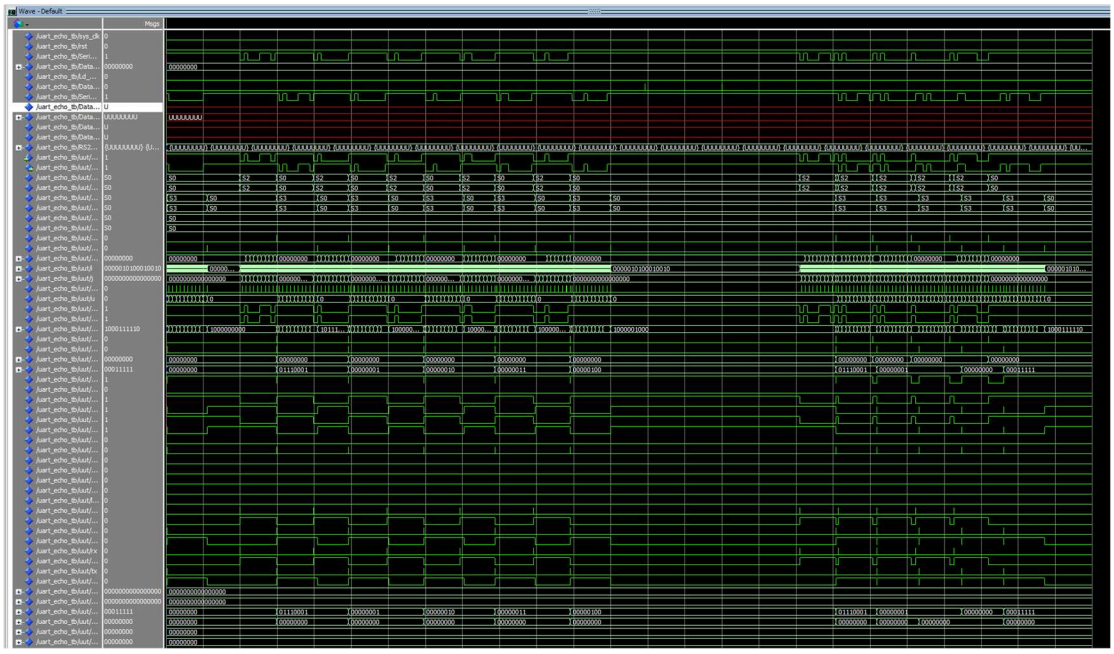

Figure 1. Serial Echo Waveform

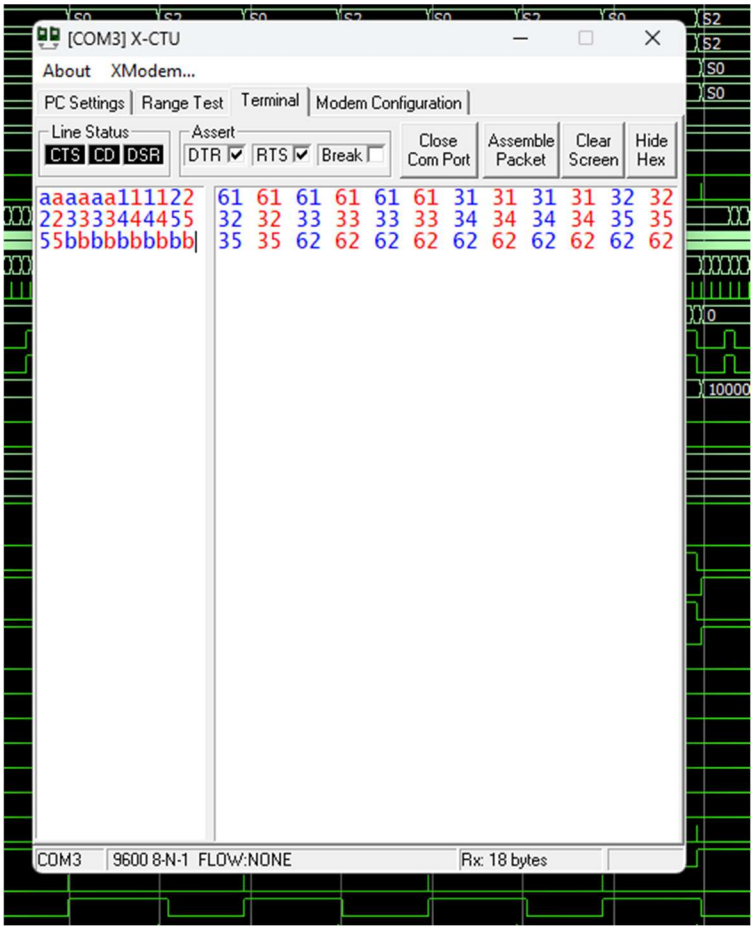

Figure 2. Serial Echo Terminal

Appendix A (Echo Chip):

```
----------------------------------------------------------------------------------
-- Company: University of Arkansas (RIOT)
-- Engineer: Tyler Cash
--
-- Create Date: 13Mar2026
-- Last Updated: 16Mar2026
-- Design Name: Serial Echo
-- Module Name: Echo Chip
-- Project Name: Echo Chip
-- Target Devices: LCMXO3D-9400HC-5BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- Top module for implementing Serial Echo Program
--
---- PinOut:
-- Signal          CPLD_Pin Description
--I/O
-- UART_TXD_IN                 A11     (Input)
-- UART_RXD_OUT     C11     (Output)
--
-- Revisions:--
--
-- Revision 0.01 -
-- File Created;
--
--
-- Additional Comments:
--
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;
use IEEE.numeric_std.all;
use IEEE.std_logic_arith.all;

library machxo3d;
use machxo3d.all;

entity UART_Echo_Top is
```

```
	generic(
		Baud : integer := 9600;
		clk_in : integer := 24930000
		);
	Port ( UART_TXD_IN : in STD_LOGIC;
		  UART_RXD_OUT : out STD_LOGIC
		  );
end UART_Echo_Top;

architecture Behavior of UART_Echo_Top is
type state_type is (S0,S1,S2,S3);
signal CS_RS232_R, NS_RS232_R, CS_RS232_W, NS_RS232_W, CS_FIFO_Bus, NS_FIFO_Bus: state_type;
signal rx_done, tx_done :STD_LOGIC:= '0';
signal temp_rcv: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
signal i, j: STD_LOGIC_VECTOR (15 downto 0):= (others => '0');
signal uartclk : STD_LOGIC:= '0';
signal u: integer;
signal UART_TXD_IN_s, UART_TXD_IN_t: STD_LOGIC:= '1';
signal txbuff: STD_LOGIC_VECTOR(9 downto 0):= (others => '1'); --buff used to transmit 1 bytes width


--Declare signals for FIFO Serial Read
signal STD_FIFO_R_WriteEn, STD_FIFO_R_ReadEn: STD_LOGIC:= '0';
signal STD_FIFO_R_DataIn, STD_FIFO_R_DataOut: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
signal STD_FIFO_R_Empty, STD_FIFO_R_Full: STD_LOGIC:= '0';


-- Declare Signals for Registers
signal LD_busy, LD_busy2, LD_rx, LD_tx, LD_temp_data, LD_temp2: STD_LOGIC:= '0';
signal LD_Temp_Addr_High, LD_Temp_Addr_Low, LD_Temp_Data_High: STD_LOGIC:= '0';
signal LD_Temp_Data_Low, ld_temp_cmd: STD_LOGIC:= '0';
signal busy, busy_reg_o, busy2, busy2_reg_o, rx, rx_reg_o, tx, tx_reg_o: STD_LOGIC:= '0';
signal temp_data_reg_o, temp_data: STD_LOGIC_VECTOR(15 downto 0):= (others => '0');
signal temp2_reg_o, temp2: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
signal Temp_Addr_High_reg_o, Temp_Addr_High: STD_LOGIC_VECTOR( 7 downto 0):= (others => '0');
signal Temp_Addr_Low_reg_o, Temp_Addr_Low: STD_LOGIC_VECTOR( 7 downto 0):= (others => '0');
signal Temp_Data_High_reg_o, Temp_Data_High: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');
```

```
signal Temp_Data_Low_reg_o, Temp_Data_Low: STD_LOGIC_VECTOR(7 downto 0):= (others =>
'0');
signal Temp_Cmd_reg_o, Temp_Cmd: STD_LOGIC_VECTOR(7 downto 0):= (others => '0');

--Internal Clock
signal OSC_Stdby, OSC_Out, OSC_SEDSTDBY, clk: std_logic := '0';

-- Reset
signal PLL_Lock, System_rst: std_logic := '0';
signal Reset_Cnt_INC, Reset_Cnt_rst: std_logic := '0';
signal Reset_Cnt_out: std_logic_vector(7 downto 0):= (others => '0');

        -- User Defined Variables
        -- CM is the Clock Divider 25MHz/CM=115200 Baud
        constant CM : integer := clk_in/Baud;
        -- CN is the read offset for erial input
        constant CN : integer :=CM/2;
        --End User Defined Variables

        -- Interal Oscillator
        COMPONENT OSCJ
        GENERIC (NOM_FREQ: string := "8.31");
        PORT ( STDBY : IN std_logic;
                        OSC : OUT std_logic;
                        SEDSTDBY : OUT std_logic
                        );
        END COMPONENT;

        -- Declare PLL
        COMPONENT PLL
        PORT (
                ClkI: in std_logic;
                ClkOP: out std_logic;
                Lock: out std_logic
                );
        END COMPONENT;

        --Declare STD_Counter Component
        component STD_Counter is
        generic
        (
                Width : integer          --width of counter
        );
        port(INC, rst, clk: in std_logic;
```

```
                Count: out STD_LOGIC_VECTOR(Width-1 downto 0));
    end component;

    --declare STD_FIFO
    COMPONENT STD_FIFO
    Generic  (
        DATA_WIDTH      :   integer;    --Width of FIFO
        FIFO_DEPTH      :   integer;    --Depth of FIFO
        FIFO_ADDR_LEN   :   integer     --Required number of bits to
represent FIFO_Depth
    );
    Port (
        CLK         : in STD_LOGIC;
        RST         : in STD_LOGIC;
        WriteEn     : in STD_LOGIC;
        DataIn : in STD_LOGIC_Vector (7 downto 0);
        ReadEn      : in STD_LOGIC;
        DataOut     : out STD_LOGIC_VECTOR (7 downto 0);
        Empty  : out STD_LOGIC;
        Full    : out STD_LOGIC
    );
    end component;

begin
    --Instantiate Internal Oscillator
    Int_OSC: OSCJ PORT MAP (
                    STDBY => OSC_Stdby,
                    OSC => OSC_Out,
                    SEDSTDBY => OSC_SEDSTDBY
                );

    --Instantiate PLL
    PLL_1: PLL PORT MAP (
                    CLKI => OSC_Out,
                    CLKOP => clk,
                    Lock => Pll_Lock
                );

    --Instantiate Reset_Cnt_8
    Reset_Cnt: Std_Counter
    generic MAP
    (
        Width => 8
    )
```

```
	port map(
		clk => OSC_Out,
		rst => Reset_Cnt_rst,
		INC => Reset_Cnt_INC,
		Count => Reset_Cnt_out
	);

	--Instantiate STD_FIFO for Reading Serial Dataa
	STD_FIFO_R: STD_FIFO
	Generic MAP
	(
		Data_Width => 8,		--Width of FIFO
		FIFO_DEPTH => 256,		--Depth of FIFO
		FIFO_ADDR_LEN => 8		--Required number of bits to represent
FIFO_DEPTH
	)
	Port MAP
	(
		CLK => clk,
		RST => System_rst,
		WriteEn => STD_FIFO_R_WriteEn,
		DataIn => STD_FIFO_R_DataIn,
		ReadEn => STD_FIFO_R_ReadEn,
		DataOut => STD_FIFO_R_DataOut,
		Empty => STD_FIFO_R_Empty,
		Full => STD_FIFO_R_Full
	);

	--Instantiate STD_FIFO for WRiting Serial Data
	--STD_FIFO_W: STD_FIFO
	--Generic MAP
	--(
		--DATA_WIDTH		=> 8,	--Width of FIFO
		--FIFO_DEPTH => 256,--Depth of FIFO
		--FIFO_ADDR_LEN	=> 8,	--Required number of bits to represent
FIFO_DEPTH
	--)
	--Port Map(
		--CLK => clk,
		--RST => rst,
		----WriteEn => STD_FIFO_W_WriteEn,
		--WriteEn => Ld_Data_TX,
		----DataIn => STD_FIFO_W_DataIn,
		--DataIn => Data_TX,
```

```
		--ReadEn => STD_FIFO_W_ReadEn,
		--DataOut => STD_FIFO_W_EMPTY,
		--Empty => STD_FIFO_W_EMPTY,
		----Full => STD_FIFO_W_Full
		--);

	----Registers
	Reg_Proc: process
	begin
		wait until clk'event and clk = '1';
		if System_rst = '0' then
			busy_reg_o <= '0';
			busy2_reg_o <= '0';
			rx_reg_o <= '0';
			tx_reg_o <= '0';
			temp_data_reg_o <= (others => '0');
			temp2_reg_o <= (others => '0');
			Temp_Addr_High_reg_o <= (others => '0');
			Temp_Addr_Low_reg_o <= (others => '0');
			Temp_Data_High_reg_o <= (others => '0');
			Temp_Data_Low_reg_o <= (others => '0');
			Temp_Cmd_reg_o <= (others => '0');

		else
		if (LD_busy = '1') then busy_reg_o <= busy; end if;
		if (LD_busy2 = '1') then busy2_reg_o <= busy2; end if;
		if (LD_rx = '1') then rx_reg_o <= rx; end if;
		if (LD_tx = '1') then tx_reg_o <= tx; end if;
		if (LD_temp_data = '1') then temp_data_reg_o <= temp_data; end if;
		if (LD_temp2 = '1') then temp2_reg_o <= temp2; end if;
		if (LD_Temp_Addr_High = '1') then Temp_Addr_High_reg_o <=
Temp_Addr_High; end if;
		if (LD_Temp_Addr_Low = '1') then Temp_Addr_Low_reg_o <= Temp_Addr_Low;
end if;
		if (LD_Temp_Data_High = '1') then Temp_Data_High_reg_o <= Temp_Data_High;
end if;
		if (LD_Temp_Data_Low = '1') then Temp_Data_Low_reg_o <= Temp_Data_Low;
end if;
		if (ld_temp_Cmd = '1') then Temp_Cmd_reg_o <= Temp_Cmd; end if;
	end if;

	end process;
	--End Registers
```

```
	---UARt Clock Divider
	UART_Clk: process
	begin
		wait until clk'event and clk = '1';
		--Synchronize async signal to rpevent metastable condition
		UART_TXD_IN_t <= UART_TXD_IN;		--Synchrol UART_TXD_IN
		UART_TXD_IN_s <= UART_TXD_IN_t;		-- Synchrol2 UART_TXD_IN

		if (System_rst = '0' or (busy_reg_o = '0' and busy2_reg_o = '0')) then
			uartclk <= '0';
			i <= CONV_STD_LOGIC_VECTOR(CN,16);	-- Set "i" to CM/2 to read at
the half point by utility
		elsif(i = CM) then
			uartclk <= '1';
			i <= X"0000";
		else
			i <= i+1;
			uartclk <= '0';
			end if;
	end process;
	---End UART Clock Divider

	---UART Read
	UART_Read: process
	begin
		wait until clk'event and clk = '1';
		if System_rst = '0' or rx_reg_o = '0' then
			temp_rcv <= x"00";
			j <= x"0000";
			rx_done <= '0';
		elsif rx_reg_o = '1' then
			if uartclk = '1' then
				if j<X"09" then
					temp_rcv(7) <= UART_TXD_IN_s;
					temp_rcv(6 downto 0) <= temp_rcv(7 downto 1);
					j <= j+1;
					rx_done <= '0';
				else
					j <= X"0000";
					rx_done <= '1';
				end if;
			else
				rx_done <= '0';
			end if;
```

```
		end if;
	end process;
	--- END UART_Read

	--- UART_Xmit
	UART_Xmit: process
	begin
		wait until clk'event and clk = '1';
		if (System_rst = '0' or tx_reg_o = '0') then
			UART_RXD_OUT <= '1';
			tx_done <= '0';
			u<=0;

			--structure the 10-bit frame to be sent
			txbuff(9)<='1'; --stopbit
			txbuff(8 downto 1) <= temp2_reg_o;
			txbuff(0)<='0'; --startbit
		else
			if uartclk = '1' then
				if(u<10) then
					UART_RXD_OUT <= txbuff(0);
					txbuff(8 downto 0) <= txbuff(9 downto 1);
					tx_done<='0';
					u<=u+1;
				else
					u<=0;
					tx_done <= '1';
				end if;
			end if;
		end if;
	end process;
	--- END UART_Xmit

	--- Next State Logic for Serial Interface Read
	NSL_RS232_R:	process(CS_RS232_R,	UART_TXD_IN_s,	rx_done,	STD_FIFO_R_Full,
temp_rcv)
	begin

		--Default states to remove latches
		busy <= '0';
		rx <= '0';
		NS_RS232_R <= S0;
		LD_busy <= '0';
		LD_rx <= '0';
```

```
        --Signals for FIFO
        STD_FIFO_R_WriteEn <= '0';
        STD_FIFO_R_DataIN <= (others => '0');


        case CS_RS232_R is
                when S0 =>
                        if (UART_TXD_IN_s = '1') then
                                NS_RS232_R <= S0;
                        else
                                NS_RS232_R <= S1;
                        end if;
                                busy <= '0';
                                rx <= '0';
                                LD_rx <= '1';
                                LD_busy <= '1';

                when S1 =>
                        NS_RS232_R <= S2;
                        busy <= '1';
                        rx <= '1';
                        LD_rx <= '1';
                        LD_busy <= '1';

                when S2 =>
                        if (rx_done = '0') then
                                NS_RS232_R <= S2;
                        else
                                NS_RS232_R <= S3;
                        end if;

                when S3 =>
                        if (STD_FIFO_R_Full = '0') then
                                STD_FIFO_R_DataIn <= temp_rcv;
                                STD_FIFO_R_WriteEn <= '1';
                        end if;
                        NS_RS232_R <= S0;

                when others =>
                        NS_RS232_R <= S0;
        end case;
end process;
---- ENd Next State Logic for Serial Interface Read
```

```
	--Next State Logic for SErial Interface Write
	NSL_RS232_W:	process(CS_RS232_W,	tx_Done,	STD_FIFO_R_Empty,
STD_FIFO_R_DataOut)
	begin

		---Default States to remove latches
		tx<='0';
		NS_RS232_W <= S0;
		temp2 <= (others => '0');
		LD_tx <= '0';
		LD_temp2 <= '0';
		Busy2 <= '0';
		LD_Busy2 <= '0';
		STD_FIFO_R_ReadEn <= '0';
		--STD_FIFO_W_ReadEn <= '0';
		--Signals for FIFO


		case CS_RS232_W is

			when S0 =>
				if(STD_FIFO_R_Empty = '1') then
					NS_RS232_W <= S0;
				else
					NS_RS232_W <= S1;
					STD_FIFO_R_ReadEn <= '1';		--Request Data from
Read FIFO
				end if;
				busy2 <= '0';
				tx <= '0';
				LD_tx <= '1';
				LD_busy2 <= '1';

			when S1 =>
				temp2<=STD_FIFO_R_DataOut;
				LD_temp2<='1';
				NS_RS232_W<=S2;

			when S2 =>
				busy2 <= '1';
				tx <= '1';
				LD_tx<='1';
				LD_busy2 <= '1';
```

```
                NS_RS232_W <= S3;

            when S3 =>
                if (tx_done ='0') then
                    NS_RS232_W <= S3;
                else
                    NS_RS232_W <= S0;
                end if;

            when others =>
                NS_RS232_W <= S0;

            end case;
        end process;
        -- End Next State Logic for Serial Interface Write


        --State Sync
        Sync_States: process
        begin
            wait until clk'event and clk = '1';
            if System_rst = '0' then
                CS_RS232_R <= S0;
                CS_RS232_W <= S0;
                CS_FIFO_BUS <= S0;
            else
                CS_RS232_R <= NS_RS232_R;
                CS_RS232_W <= NS_RS232_W;
                CS_FIFO_BUS <= NS_FIFO_Bus;
            end if;
        end process;
        --End State Sync

        --Reset Block1
Reset_Blk1: process
begin
        wait until OSC_Out'event and OSC_Out = '1';
            if(PLL_Lock = '0') then
                Reset_Cnt_rst <= '0'; -- Active Low
            else
                Reset_Cnt_rst <= '1';
            end if;
end process;
```

```
		--Reset Block
	Reset_Blk: process
	begin
		wait until OSC_Out'event and OSC_Out ='1';
			if(Reset_Cnt_out < X"7F") then
				System_rst <= '0'; --Active Low
				Reset_Cnt_INC <= '1';
			else
				System_rst <= '1';
				Reset_Cnt_INC <= '0'; -- ACtive Low
			end if;
	end process;

end Behavior;

----------------------Generic FIFO----------------------
Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;


entity STD_FIFO is
	Generic (
		DATA_WIDTH	:	integer := 8;		--Width of FIFO
		FIFO_DEPTH	:	integer := 256;		--Depth of FIFO
		FIFO_ADDR_LEN	:	integer := 9		--Required number of bits to
represent FIFO_Depth
	);
	Port (
		CLK			:	in	STD_LOGIC;
		RST			:	in	STD_LOGIC;
		WriteEn			:	in	STD_LOGIC;
		DataIn		:	in	STD_LOGIC_VECTOR	(Data_WIDTH	-	1
downto 0);
		ReadEn			:	in	STD_LOGIC;
		DATAOUT			:	out STD_LOGIC_VECTOR (DATA_WIDTH - 1
downto 0);
		Empty		:	out STD_LOGIC;
		Full		:	out STD_LOGIC
	);
end STD_FIFO;

architecture Behavioral of STD_FIFO is
```

```
	type FIFO_Memory is array (0 to FIFO_DEPTH - 1) of STD_LOGIC_VECTOR (DATA_WIDTH
- 1 downto 0);
	signal Memory : FIFO_MEmory;
	signal Head : STD_LOGIC_VECTOR (FIFO_ADDR_LEN-1 downto 0);
	signal Tail : STD_LOGIC_VECTOR (FIFO_ADDR_LEN-1 downto 0);
	signal Looped : boolean;
begin

	--Memory Pointer Process
	fifo_proc : process (CLK)

	begin
		if rising_edge(CLK) then
			if RST = '0' then
				Head <= (others => '0');
				Tail <= (others => '0');
				DataOut <= (others => '0');
				Looped <= false;
				Full <= '0';
				Empty <= '0';
			else
				if(ReadEn = '1') then
					if ((Looped = true) or (Head /= Tail)) then
					--Update data output
						DataOut <= Memory(CONV_INTEGER(Tail));

						-- Update Tail pointer as needed
						if (Tail = FIFO_DEPTH -1) then
							Tail <= (others => '0');

							Looped <= false;
						else
							Tail <= Tail + 1;
						end if;
					end if;
				end if;

				if (WriteEn = '1') then
					if ((Looped = false) or (Head /= Tail)) then
						-- Write Data to Memory
						Memory(CONV_INTEGER(Head)) <= DataIn;
						-- Increment Head point as needed
						if (Head = FIFO_DEPTH -1) then
```

```
                                        Head <= (others => '0');
                                        Looped <= true;
                                    else
                                        Head <= Head + 1;
                                    end if;
                                end if;
                            end if;

                            --Update Empty and Full flags
                            if (Head = Tail) then
                                if Looped then
                                    Full <= '1';
                                else
                                    Empty <= '1';
                                end if;
                            else
                                Empty <= '0';
                                Full <= '0';
                            end if;
                        end if;
                end if;
        end process;

end Behavioral;

--------- End Generic FIFO ----------------
Library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;

entity Std_Counter is
        generic
        (
                Width : integer := 8
        );
        port(INC, rst, clk: in std_logic;
                Count: out STD_LOGIC_VECTOR(Width -1 downto 0));
end;

architecture BEHAVIOR of Std_Counter is
signal temp: STD_LOGIC_VECTOR(Width - 1 downto 0);
begin
        Counter_behav: process
```

```
	begin
		wait until clk'event and clk = '1';
		if rst = '0' then
			temp <= (Others => '0');
		elsif INC = '1' then
			temp <= temp + 1;
			else
		null;
		end if;

	end process;
	Count <= temp;
end BEHAVIOR;
```

Appendix B (Echo TB):

```
----------------------------------------------------------------------------------
-- Company: University of Arkansas
-- Engineer: Tyler Cash
--
-- Create Date: 03/25/2026
-- Design Name: Echo Chip
-- Module Name: UART_Echo_tb - Behavioral
-- Project Name: HW7
-- Target Devices: LCMXO3D-9400HC-5BG256C (MachXO3D_BreakoutBrd)
-- Tool versions: Lattice Diamond_x64 Build 3.12.0.240.2
-- Description:
-- TB for Serial Echo Program
--
---- PinOut:
-- Signal          CPLD_Pin Description
--I/O
-- UART_TXD_IN                A11     (Input)
-- UART_RXD_OUT     C11     (Output)
--
-- Revisions:--
--
-- Revision 0.01 -
-- File Created;
--
--
-- Additional Comments:
--
--
----------------------------------------------------------------------------------
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use ieee.numeric_std.all;

entity UART_Echo_tb is
end UART_Echo_tb;

architecture Behavioral of UART_Echo_tb is
```

```
 -- Component Declaration for the Unit Under Test (UUT)

 COMPONENT UART_Echo_Top
  Port (          UART_TXD_IN : in STD_LOGIC;
                  UART_RXD_OUT : out STD_LOGIC
      );
 END COMPONENT;


--Inputs
signal sys_clk : std_logic := '0';
signal rst : std_logic := '0';
signal Serial_RX : std_logic := '0';
signal Data_TX : STD_LOGIC_VECTOR (7 downto 0) := X"00";
signal Ld_Data_TX : std_logic := '0';
signal Data_Rqst : std_logic := '0';

  --Outputs
signal Serial_TX : std_logic;
signal Data_TX_Full: std_logic;
signal Data_RX : STD_LOGIC_VECTOR (7 downto 0);
signal Data_RX_Empty : std_logic;
signal Data_Rdy : std_logic;

-- Clock period definitions
 constant clk_period : time := 40 ns; --[25 MHz ish]
  --constant read_Time: time :=8680 ns; --for 115,200 Baud
 constant read_Time: time :=104100 ns; --for 9,600 Baud [100 MHz Clock]
 --constant read_Time: time :=2597 ns; --for 9,600 Baud [24.93 MHz Clock]    (Clk_In/Baud *
clk_period)

type Memory is array (255 downto 0) of STD_LOGIC_VECTOR (7 downto 0);
signal RS232_Cmd: Memory;

BEGIN

  -- Instantiate the Unit Under Test (UUT)
 uut: UART_Echo_Top PORT MAP (
        UART_TXD_IN => Serial_RX,
        UART_RXD_OUT => Serial_TX
     );
```

```
----Define Command Memory
--Write Enable Command
RS232_Cmd(0) <= X"71";          --Cmd (Write)
RS232_Cmd(1) <= X"01";          --Address High
RS232_Cmd(2) <= X"02";          --Address Low
RS232_Cmd(3) <= X"03";          --Data Byte1
RS232_Cmd(4) <= X"04";          --Data Byte2

--Write VCmd to 0x0FFF
RS232_Cmd(5) <= X"71";          --Cmd (Write)
RS232_Cmd(6) <= X"01";          --Address High
RS232_Cmd(7) <= X"01";          --Address Low
RS232_Cmd(8) <= X"0F";          --Data Byte1
RS232_Cmd(9) <= X"FF";          --Data Byte2

--Write VCmd to 0x001F
RS232_Cmd(10) <= X"71";          --Cmd (Write)
RS232_Cmd(11) <= X"01";          --Address High
RS232_Cmd(12) <= X"01";          --Address Low
RS232_Cmd(13) <= X"00";          --Data Byte1
RS232_Cmd(14) <= X"1F";          --Data Byte2

--Write ICmd to 0xFFFF
RS232_Cmd(15) <= X"71";          --Cmd (Write)
RS232_Cmd(16) <= X"01";          --Address High
RS232_Cmd(17) <= X"02";          --Address Low
RS232_Cmd(18) <= X"FF";          --Data Byte1
RS232_Cmd(19) <= X"FF";          --Data Byte2

--Read VCmd Register
RS232_Cmd(20) <= X"70";          --Cmd (Read)
RS232_Cmd(21) <= X"01";          --Address High
RS232_Cmd(22) <= X"01";          --Address Low




---- Clock process definitions
--SYSCLK_process :process
--begin
    --sys_clk <= '0';
    --wait for clk_period/2;
    --sys_clk <= '1';
```

```
    --wait for clk_period/2;
 --end process;



 -- Stimulus process
 stim_proc: process
 begin
    -- initialize serial ports to idle state
    Serial_RX <= '1';

   -- hold reset state for 100 ns.
--     rst <='0';
-- wait for clk_period*10;
--     rst <='1';
--     wait for clk_period*256;
   -- insert stimulus here

    wait for 2 ms;


    --Write Enable Command
    Serial_RX <= '0';               --Send Start Bit
    wait for read_Time;
    for i in 0 to 7 loop
       Serial_RX <= RS232_Cmd(0)(i);
       wait for read_Time;
    end loop;
    Serial_RX <= '1';               --Send Stop Bit
    wait for read_Time*10;

    Serial_RX <= '0';               --Send Start Bit
    wait for read_Time;
    for i in 0 to 7 loop
       Serial_RX <= RS232_Cmd(1)(i);
       wait for read_Time;
    end loop;
    Serial_RX <= '1';               --Send Stop Bit
    wait for read_Time*10;

    Serial_RX <= '0';               --Send Start Bit
    wait for read_Time;
    for i in 0 to 7 loop
       Serial_RX <= RS232_Cmd(2)(i);
```

```
   wait for read_Time;
end loop;
Serial_RX <= '1';              --Send Stop Bit
wait for read_Time*10;

Serial_RX <= '0';              --Send Start Bit
wait for read_Time;
for i in 0 to 7 loop
   Serial_RX <= RS232_Cmd(3)(i);
   wait for read_Time;
end loop;
Serial_RX <= '1';              --Send Stop Bit
wait for read_Time*10;

Serial_RX <= '0';              --Send Start Bit
wait for read_Time;
for i in 0 to 7 loop
   Serial_RX <= RS232_Cmd(4)(i);
   wait for read_Time;
end loop;
Serial_RX <= '1';              --Send Stop Bit
wait for read_Time*10;
--End Write Test

wait for read_Time*10;


-----Request two samples from FIFO------
Data_Rqst <= '1';
wait for clk_period;
Data_Rqst <= '0';
wait for read_Time*20;

Data_Rqst <= '1';
wait for clk_period;
Data_Rqst <= '0';
wait for read_Time*20;

--------End Request-------

--Write Voltage Command
Serial_RX <= '0';              --Send Start Bit
wait for read_Time;
for i in 0 to 7 loop
```

```
    Serial_RX <= RS232_Cmd(10)(i);
    wait for read_Time;
  end loop;
  Serial_RX <= '1';              --Send Stop Bit
  wait for read_Time;

  Serial_RX <= '0';              --Send Start Bit
  wait for read_Time;
  for i in 0 to 7 loop
    Serial_RX <= RS232_Cmd(11)(i);
    wait for read_Time;
  end loop;
  Serial_RX <= '1';              --Send Stop Bit
  wait for read_Time;

  Serial_RX <= '0';              --Send Start Bit
  wait for read_Time;
  for i in 0 to 7 loop
    Serial_RX <= RS232_Cmd(12)(i);
    wait for read_Time;
  end loop;
  Serial_RX <= '1';              --Send Stop Bit
  wait for read_Time;

  Serial_RX <= '0';              --Send Start Bit
  wait for read_Time;
  for i in 0 to 7 loop
    Serial_RX <= RS232_Cmd(13)(i);
    wait for read_Time;
  end loop;
  Serial_RX <= '1';              --Send Stop Bit
  wait for read_Time;

  Serial_RX <= '0';              --Send Start Bit
  wait for read_Time;
  for i in 0 to 7 loop
    Serial_RX <= RS232_Cmd(14)(i);
    wait for read_Time;
  end loop;
  Serial_RX <= '1';              --Send Stop Bit
  wait for read_Time;
  --End Write Test

wait;
```

```
 end process;

end;
```